Phase 3

Single-Cycle RV32I CPU

EEL 4768 Computer Architecture
Fall 2026

Learning Objectives

By the end of this phase you should be able to:

- Assemble the blocks you built in Phase 2 into one working datapath.
- Design the full datapath of a single-cycle RV32I CPU.
- Validate a large, complex, multi-module design.

Note

You must write your own Verilog test benches to verify your designs. An example test trace will be provided, as well as a skeleton for `hart.v` in `phase_3/skeletons/`.
You also should reference the RV32I reference card from webcourses for the instruction formats, opcodes, and immediate values.

1 Submission

- **One submission per group.** You will manually submit your work to Webcourses as a zip file. Students in multiple groups, or not in a group by the time of submission, will receive **no credit** for the submission.
- **Files to submit**:

```
hart.v
alu.v
imm.v
rf.v
decoder.v
```

One module per file, and name the module the same as the filename. The testbenches instantiate modules named `hart`, `alu`, `imm`, `rf` and `decoder` by name.

2 Grading Rubric

Category	Points
Single-Cycle CPU	5.0
Total	**5.0**

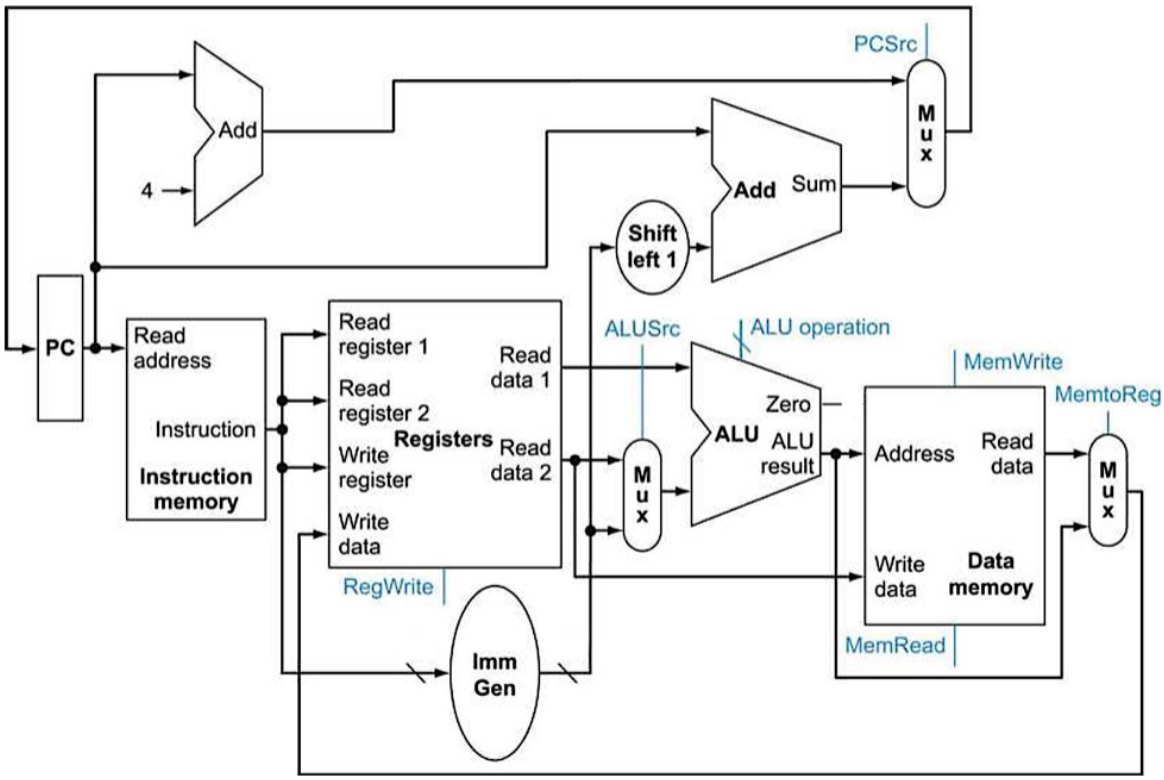
3 Verilog Coding Rules

Your submission is internally checked against a synthesizable subset. This is to insure your code will be synthasizable for future phases. The rules listed below will result in a 0 for the specific file.

Rule	**What it means for you**
No `*`, `/`, `%`, `**`	Arithmetic that does not synthesize cheaply. You do not need any of them in this phase.
No `===`, `!==`, `==?`, `!=?`	Not synthesizable. Use `==` and `!=`.
Shifts only by a constant	`x << 3` is fine; `x << i_op2[4:0]` is *rejected*.
No `if`/`else` in combinational logic	`if`/`else` is only legal inside a clocked `always @(posedge ...)` block. For combinational logic use a continuous `assign` with the `?:` operator, or a `case` inside `always @(*)` (Valid for generate blocks on parameters and sequential logic only).
No procedural loops	`for`, `while`, `repeat` and `do`/`while` statements are rejected. To build something repetitive, use a `generate` loop, or just write the instances out.
No delays or waits	`#5`, `wait`, `disable` and friends are simulation constructs.
No system tasks	`$display`, `$finish`, `$stop`, `$system`, `$readmemh`/`$readmemb`, `$writememh`/`$writememb` and all file I/O (`$fopen`, `$fwrite`, ...) are not allowed in your final submission. Only constant functions such as `$clog2` are allowed.
No `` `include ``	Not needed for importing other files.

4 Single-Cycle CPU

In phase 2, you designed and tested blocks that implement the ALU, register file, immediate generator and instruction decoder. In this phase, you will wire those blocks together into a single-cycle CPU that fetches, decodes, executes and writes back one instruction per clock cycle.




```
module hart #(parameter RESET_ADDR = 32'h00000000) (
    input  wire        i_clk,
    input  wire        i_rst,
    /* ... */
);
```

4.1 ALU, RF, Imm, and Decoder

These modules will be reused from phase 2, and instantiated in `hart.v`. There are some small changes to the ALU and RF, which are described in Section 4.6.

4.2 PC

In a processor, the program counter (PC) is a register that holds the address of the instruction to be fetched. You will need to implement a PC, which always points to the current instruction.

To go to the next instruction, the PC is either incremented by 4 (the size of an instruction in bytes), or set to a branch or jump target address. The PC is initialized to `RESET_ADDR` on reset.

4.3 Branch and jump

Your processor will need to additionaly handle branching and jumping logic, which is used to overwrite the PC. When a branch or jump instruction is executed, the PC will be set to the target address. Otherwise, the PC will be incremented by 4.

4.4 Instruction and data memory

Besides the program counter and branch/jump logic, your CPU will need to handle the instruction and memory interfaces. Assume that the instruction and data memories are external to your design, and reading/writing requires no latency. Both are combinational, and require no clock edge to read or write. The instruction memory is read-only, while the data memory is read/write.

Port	Dir/Width	Meaning
`o_imem_raddr`	out [31:0]	Instruction address. One instruction per cycle, always 4-byte aligned.
`i_imem_rdata`	in [31:0]	The instruction word at `o_imem_raddr`.
`o_dmem_addr`	out [31:0]	Data address. **Always word-aligned.**
`o_dmem_ren`	out	Read enable. Never high in the same cycle as `o_dmem_wen`.
`o_dmem_wen`	out	Write enable.
`o_dmem_wdata`	out [31:0]	Store data. Only the lanes selected by `o_dmem_mask` are used.
`o_dmem_mask`	out [3:0]	Which of the four byte lanes of the word at `o_dmem_addr` are read or written.
`i_dmem_rdata`	in [31:0]	The full word at `o_dmem_addr`, regardless of mask. Extracting and sign- or zero-extending the right bytes is `hart.v`'s job.

Clarification

Sub-word accesses use the mask, not the address. To pass a 32-bit word, all four lanes of `o_dmem_mask` must be high. See the comments on the `hart.v` skeleton for more details.

4.5 Retire interface

These outputs are not part of RV32I. They exist for the testbench to validate your design and its functionality. Set each signal appropriately every cycle. Because your design is single-cycle, every fetched instruction retires in the cycle it is fetched: `o_retire_valid` is high every cycle after `i_rst` deasserts, with no bubbles, through the cycle `o_retire_halt` fires.

Port	Width	Meaning
`o_retire_valid`	1	An instruction retired this cycle.
`o_retire_inst`	[31:0]	The raw fetched instruction word.
`o_retire_trap`	1	Illegal encoding, or a misaligned access.
`o_retire_halt`	1	The instruction is `ebreak`.
`o_retire_rs1_raddr`	[4:0]	Source register 1, `5'd0` **if not read**.
`o_retire_rs1_rdata`	[31:0]	Value read from `rs1`.
`o_retire_rs2_raddr`	[4:0]	Source register 2, `5'd0` **if not read**.
`o_retire_rs2_rdata`	[31:0]	Value read from `rs2`.
`o_retire_rd_waddr`	[4:0]	Destination register, `5'd0` **if no register is written**.
`o_retire_rd_wdata`	[31:0]	Value written to `rd`.
`o_retire_pc`	[31:0]	PC this instruction was fetched from.
`o_retire_next_pc`	[31:0]	`PC + 4`, or the branch or jump target when taken.

Note

Every field above is checked on every retiring instruction, Make sure to handle these signals every cycle. Failure to do so will make your design not integrate with the testbench, and will result in a 0 for the entire phase.

4.6 Instantiating your Phase 2 modules

- `rf.v` **is instantiated with** `BYPASS_EN = 0`**.** In a single-cycle design an instruction's write-back commits on the same clock edge the next instruction's read samples, so there is no same-cycle read/write collision to bypass. Bypass will later be used in the pipelined datapath.
- `decoder.v` **already contains** `imm.v`**.** You instantiated it there in Phase 2; do not instantiate a second copy.
- `alu.v` **performs the branch comparison.** There is no separate comparator: `beq`, `bne`, `blt`, `bge`, `bltu` and `bgeu` all read the ALU's `o_eq` and `o_slt` while `o_result` is unused.

Note

Phase 3 adds a small change to the ALU and RF.

- You should additionally drive your ALU's `o_sltu` when `i_op` is `3'b011` (the `SLTU` instruction). This is the only change to your Phase 2 ALU.
- RF no longer contains `o_rd_wen`. Instead, gate instructions by setting the `i_rd_waddr` to `5'd0`.

For assistance making these changes, please come to office hours.

5 Testing Your Work

There is no autograder in this repository. Verifying that `hart.v` runs real programs correctly is part of the assignment. A trace is provided in `phase_3/traces/` that you can use to validate your design. It is one continuously executing RV32I program of about 22,600 instructions, and it comes as two files rather than Phase 2's one:

- `vectors/hart_program.hex` — the program itself, as a flat instruction memory image in `$readmemh` format, one instruction per line, with the address being the line number times four. Your design fetches from it at whatever address it computes.
- `vectors/hart.trace` — one line per instruction actually retired, in the order it happens, with one column per retire signal. Nothing in it is driven into your design; it is only compared against. A column written as `x` is a don't-care.

5.1 Running it

Put your Verilog in `phase_3/traces/submission/` and run `./run_traces.sh`, or point it at wherever your files already live:

```
$ ./run_traces.sh ~/eel4768/phase_3
...
[FAIL] vector 9645 (LOAD_STORE.SB_LB): pc=000096b0 inst=009f8123
         mem_wdata lane 3: got ff, expected 2a
...
---------- summary ----------
vectors: 22661   passed: 22600   failed: 61
  LOAD_STORE.SB_LB  300/360 <-- FAIL

TEST FAILED: 61 of 22661 vectors wrong.
```

Each failure names the signal that disagreed and the instruction it disagreed on. The full output and the compile log are left in `traces/build/hart/`. You need `iverilog` and `vvp`; nothing else.